REPORTE TAREA 1
ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Emilio Jesus Morán Valdebenito

9 de abril de 2026

20:43

1. Entorno experimental

Los experimentos fueron ejecutados en un equipo con procesador **Intel® Core™ i5-11400H de 11ª generación a 2,70 GHz (2,69 GHz efectivos)**, con **16,0 GB de RAM** (15,8 GB utilizables). El sistema operativo utilizado fue **Windows 11 / Ubuntu 24.04.4 LTS**, y el compilador empleado fue **gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) versión 13.3.0**.

Se utilizaron los conjuntos de datos descritos en el enunciado de la tarea. Para el problema de ordenamiento, se emplearon arreglos de tamaño $n \in \{10^1, 10^3, 10^5, 10^7\}$ con variaciones en su tipo (ascendente, descendente, aleatorio) y dominio ($D1, D7$). Para la multiplicación de matrices, se usaron matrices cuadradas de dimensiones $n \in \{2^4, 2^8, 2^{10}\}$ con estructuras dispersas, diagonales y densas, bajo los dominios $D0$ y $D10$. En ambos casos, se consideraron tres muestras aleatorias (a, b, c) por configuración para las mediciones experimentales.

2. Resultados y Análisis

A continuación se presentan los resultados obtenidos para los experimentos de ordenamiento y multiplicación de matrices. Cada figura incluye una breve descripción de las observaciones más relevantes.

Algoritmos de ordenamiento

La Fig. 1 muestra el tiempo de ejecución en función del tamaño de entrada n para cada algoritmo de ordenamiento evaluado. En general, todos los algoritmos escalan de manera muy similar, siendo Merge

Sort [3] el que más se demora en todos los casos, seguido por Quick Sort [2] con un tiempo inferior pero aún por encima de `std::sort`, que resultó ser el algoritmo más eficiente. Sin embargo, existen excepciones notables según el tipo de ordenamiento: cuando los datos están en orden ascendente, Merge Sort presenta un tiempo inicial muy elevado comparado con los otros algoritmos que comienzan con tiempos similares; en contraste, cuando los datos están en orden descendente, `std::sort` muestra un mejor desempeño inicial.

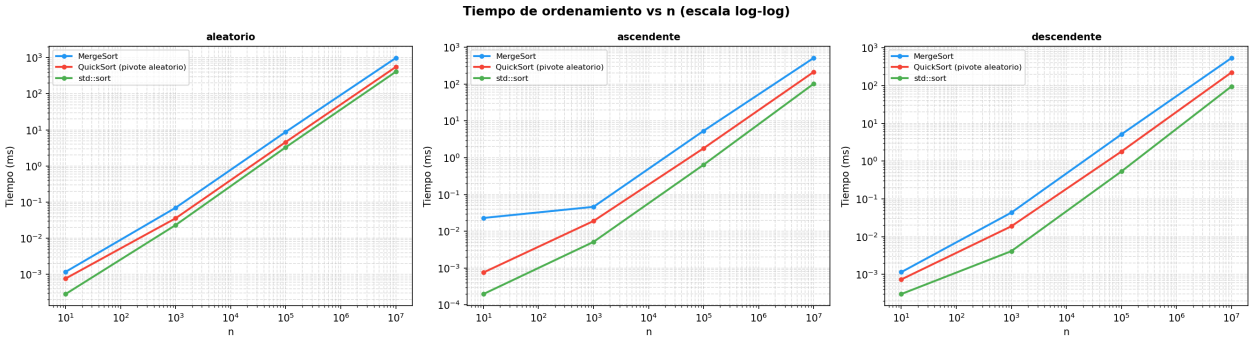

Figura 1: Tiempo de ejecución vs. tamaño de entrada n para los algoritmos de ordenamiento evaluados.

La Fig. 2 presenta el uso de memoria en función del tamaño de entrada para cada tipo de algoritmo. Se aprecia que algoritmos recursivos como Merge Sort [3] presentan un mayor consumo de memoria debido a la pila de llamadas y a la necesidad de arreglos auxiliares.

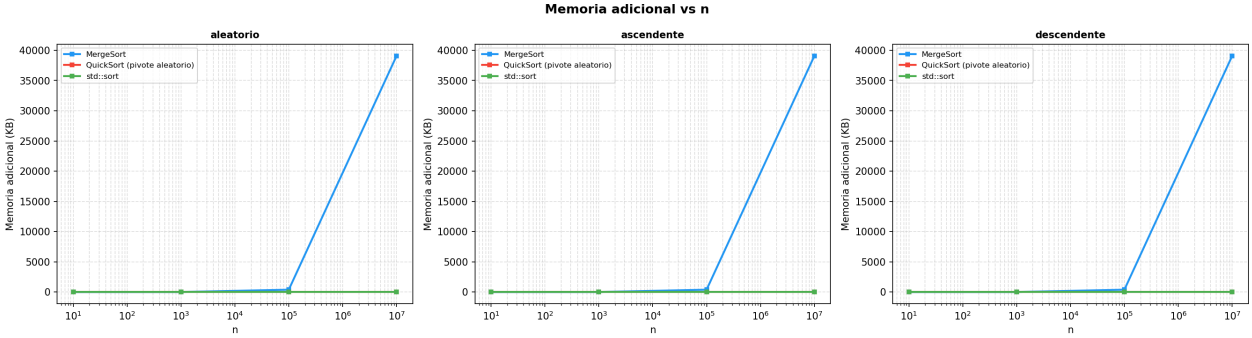

Figura 2: Uso de memoria vs. tamaño de entrada n para los algoritmos de ordenamiento, agrupados por tipo.

La Fig. 3 muestra un mapa de calor del tiempo de ejecución para $n = 100{,}000$ entre los distintos algoritmos. Esta visualización permite comparar de forma directa el rendimiento relativo de cada algoritmo en un tamaño de entrada fijo y de mayor escala.

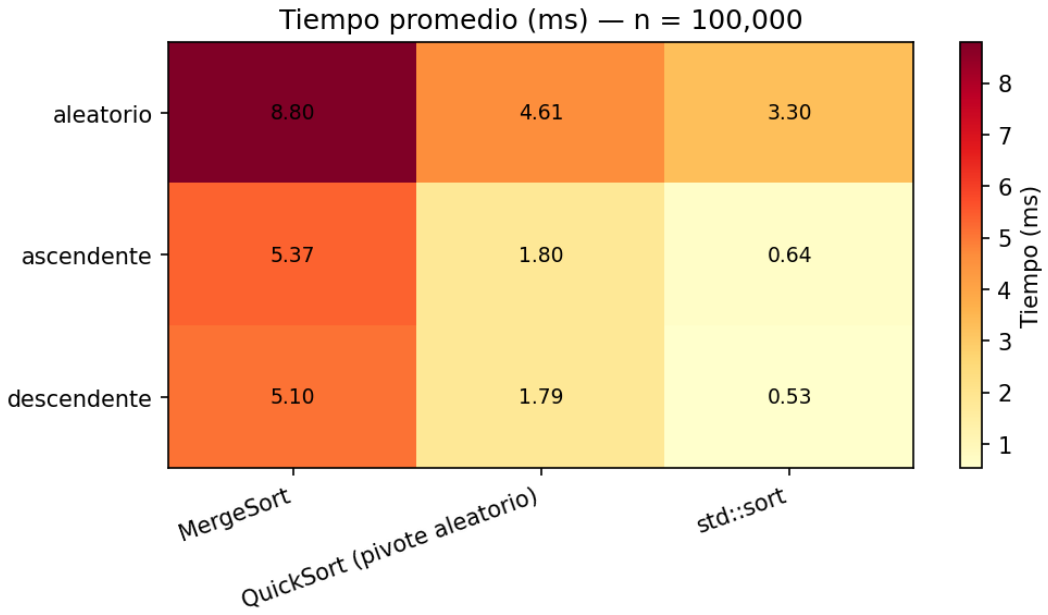

Figura 3: Mapa de calor del tiempo de ejecución para $n = 100{,}000$ entre los algoritmos de ordenamiento.

Multiplicación de matrices

La Fig. 4 presenta el tiempo de ejecución en función del tamaño de la matriz $n \times n$ para cada algoritmo de multiplicación evaluado. Se observa que el algoritmo de Strassen [4] reduce el tiempo de ejecución para matrices de mayor tamaño respecto al algoritmo cúbico clásico, confirmando su ventaja asintótica [1].

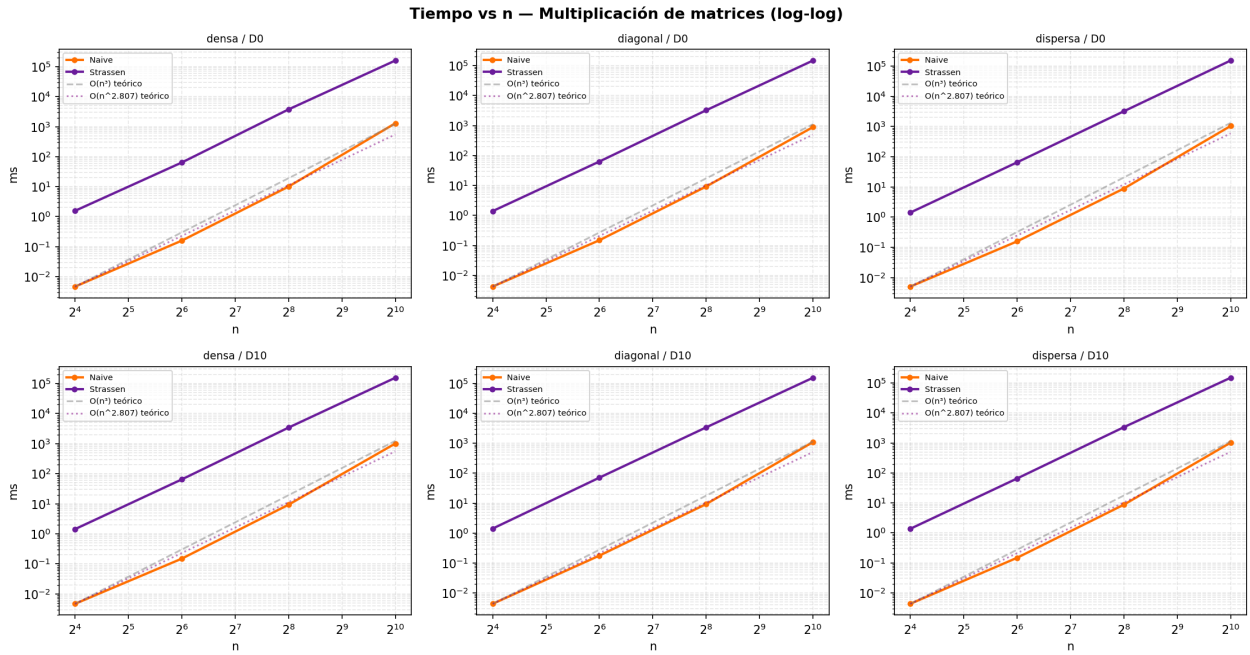

Figura 4: Tiempo de ejecución vs. tamaño de matriz n para los algoritmos de multiplicación de matrices, agrupados por tipo.

La Fig. 5 muestra el uso de memoria en función de n para cada algoritmo de multiplicación. El algoritmo de Strassen [4] exhibe un mayor consumo de memoria producto de las submatrices intermedias generadas en cada nivel de recursión, como se preveia.

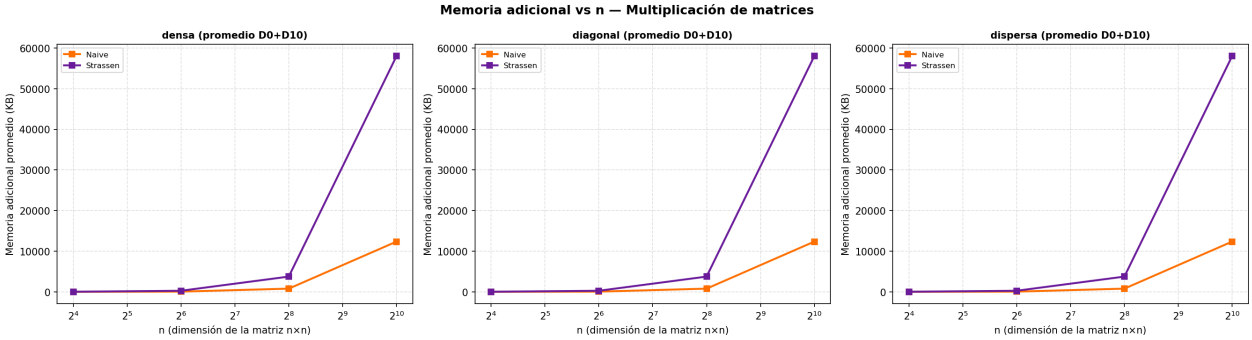

Figura 5: Uso de memoria vs. tamaño de matriz n para los algoritmos de multiplicación de matrices, por tipo.

3. Conclusiones

En base a los experimentos realizados, se concluye que tanto en algoritmos de ordenamiento como en multiplicación de matrices, el desempeño relativo de los algoritmos está fuertemente influenciado por la naturaleza de los datos de entrada. En ordenamiento, aunque todos los algoritmos escalan de manera similar, existe una clara jerarquía de eficiencia con `std::sort` como el algoritmo óptimo, seguido

de Quick Sort y Merge Sort; sin embargo, el comportamiento difiere según el orden inicial de los datos, pudiendo Merge Sort ser significativamente más lento en datos ya ordenados ascendentemente. En multiplicación de matrices, el algoritmo de Strassen confirma su ventaja asintótica para matrices grandes, reduciendo el tiempo de cómputo en comparación con el algoritmo clásico, validando así la importancia de la selección de algoritmos eficientes en operaciones computacionalmente intensivas.

Referencias

[1] Christophe Fiorio. *Algorithm2e documentation*. http://ctan.math.illinois.edu/macros/latex/contrib/algorithm2e/doc/algorithm2e.pdf. 2023.

[2] GeeksforGeeks. *3-Way QuickSort (Dutch National Flag)*. https://www.geeksforgeeks.org/dsa/3-way-quicksort-dutch-national-flag/. 2024.

[3] GeeksforGeeks. *Merge Sort*. https://www.geeksforgeeks.org/dsa/merge-sort/. 2024.

[4] GeeksforGeeks. *Strassen's Matrix Multiplication*. https://www.geeksforgeeks.org/dsa/strassens-matrix-multiplication/. 2024.